

Unit 1 · Topic Notes

Requirement Analysis & Use-Case Identification

Course: Full Stack Application Development with MERN

Learning Objectives

1. Distinguish functional vs. non-functional requirements
 2. Write clear user stories with acceptance criteria
 3. Identify actors and use cases for a web application
 4. Turn a vague client ask into a scoped requirements doc
-

1. Why This Comes Before Any Code

The #1 cause of a failed capstone project isn't bad code — it's building the wrong thing because nobody wrote down what "done" means. Requirement analysis is how a team agrees, on paper, what they're building before touching VS Code.

2. Functional vs. Non-Functional Requirements

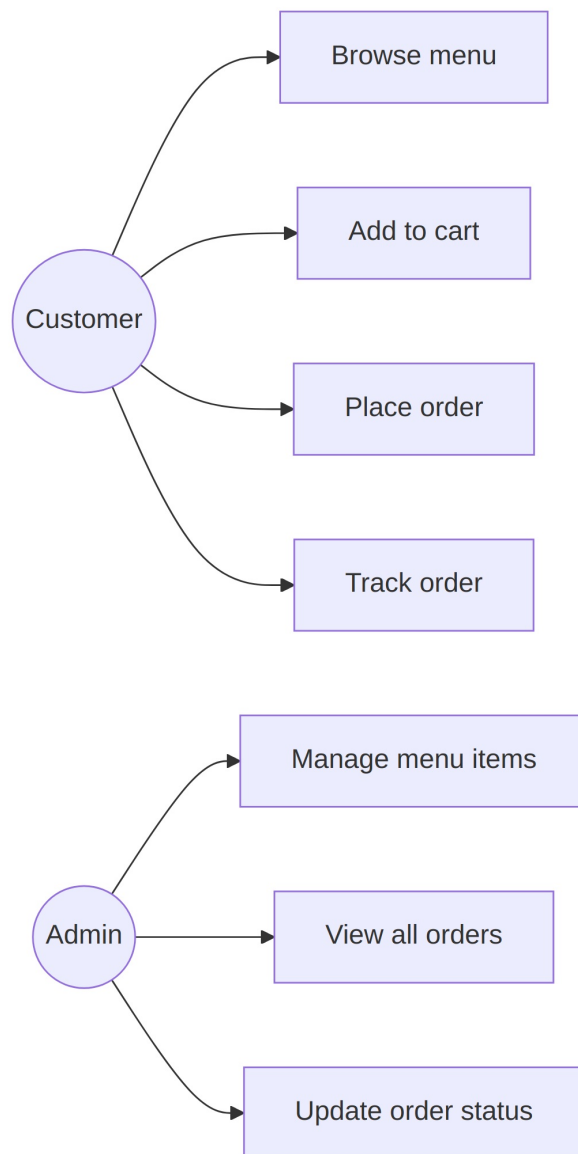
Type	Answers	Example (for a food-delivery MERN app)
Functional	What must the system <i>do</i> ?	"User can add items to cart and place an order"
Non-Functional	How well must it do it?	"Page must load in under 2 seconds", "Support 500 concurrent users", "Passwords must be hashed"

Common non-functional categories: performance, security, scalability, usability, availability.

3. Actors and Use Cases

An **actor** is anyone/anything that interacts with the system (User, Admin, Payment Gateway, Delivery Partner). A **use case** is a goal that actor wants to accomplish.

Since Mermaid doesn't render formal UML use-case diagrams, this actor→goal flowchart works just as well for a student project:



Actor and use case diagram

4. Writing User Stories

Standard format:

As a [type of user], I want [an action], so that [a benefit/reason].

Example set for a capstone “campus food ordering app”:

As a student, I want to see today's canteen menu, so that I can decide what to order.
As a student, I want to pay online, so that I don't have to carry cash.
As a canteen admin, I want to mark an order "ready", so that the student is notified.

4.1 Acceptance Criteria (Given/When/Then)

Every user story should have testable criteria before anyone writes code:

```
Story: As a student, I want to add an item to my cart.
```

```
Given I am viewing the menu  
When I click "Add to Cart" on an item  
Then the item appears in my cart with quantity 1  
And the cart icon shows an updated item count
```

This format (Gherkin-style) directly maps to what QA/testing will check later in Unit 4.

5. From Requirements to a Scope Document

A minimal one-page requirements doc for a student team should contain:

1. **Project one-liner** — “A web app for X users to do Y”
2. **Actors** (2-4)
3. **Functional requirements** — grouped by actor, as user stories
4. **Non-functional requirements** — 4-6 bullet points
5. **Out of scope** — explicitly list what you’re *not* building (this prevents scope creep, the #1 killer of semester projects)

6. Hands-On Lab (30 min, pairs)

Scenario: “Build a web app where college students can report broken campus infrastructure (broken lights, leaking taps, etc.) and admins can track repair status.”

Tasks:

1. Identify all actors (at least 2)
2. Write 6 user stories (mix of both actors) in the correct format
3. Add acceptance criteria (Given/When/Then) to 3 of them
4. Write a one-line “Out of scope” list (at least 3 items — e.g., “no push notifications in v1”)
5. Draw the actor→use-case flowchart (paper or Mermaid)

7. Quick Assessment

1. Give one functional and one non-functional requirement for a login page.
2. Rewrite this badly-written requirement as a proper user story: “Login should work.”
3. What’s the purpose of an “Out of scope” section?
4. In Given/When/Then, which part describes the *action* the user takes?
5. Name two actors you’d expect in a hostel room-booking system.

8. Common Mistakes to Avoid

- Writing requirements as implementation details (“Use MongoDB” is not a requirement, it’s a design decision)
- Skipping non-functional requirements entirely, then being surprised when the app is slow
- No “out of scope” section → feature creep two weeks before submission

Notes prepared for Full Stack Learning — MERN course, Unit 1.